

Image Processing

LAB MANUAL

Himanshu Sardana 102303244

Contents

WAP to convert grayscale images to binary images	3
WAP to convert rgb image to grayscale	4
WAP to draw a border around the input image, input the width (in terms of pixel count) and the desired color of the border from the user. Try this with a binary and grayscale image.	6
WAP for image complement (try with both, a grayscale and a binary image)	7
WAP to enhance the input image using log transform (assume $c = 1$)	8
WAP to enhance the input image using Intensity Level Slicing (Contrast Stretching)	10
WAP to enhance the input image using histogram equalization	11
WAP to match the histogram of the input image with that of reference image using histogram matching technique	12
WAP to smooth the image using Averaging filter (un-weighted), Weighted filter given by $h(x, y) = \max(x , y)$, gaussian filter	13
WAP to sharpen the input image using Laplacian filter, Gradient operators (Sobel, Canny), Unsharp masking	14

QUESTION 1

WAP to convert grayscale images to binary images

SOLUTION

```
import numpy as np
import matplotlib.pyplot as plt
import cv2

img = cv2.imread("./gray_original.png", cv2.IMREAD_GRAYSCALE)
_, binary_img = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)

plt.subplot(1, 2, 1)
plt.title("Original Image")
plt.imshow(img, cmap='gray')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.title("Binary Image")
plt.imshow(binary_img, cmap='gray')
plt.axis('off')
```

OUTPUT

(np.float64(-0.5), np.float64(699.5), np.float64(465.5), np.float64(-0.5))<Figure size 640x480 with 2 Axes>

Original Image



Binary Image



SOLUTION

```
import numpy as np
import matplotlib.pyplot as plt
import cv2

img = cv2.imread("./gray_original.png", cv2.IMREAD_GRAYSCALE)
threshold = int(input("Enter threshold value (0-255): "))
_, binary_img = cv2.threshold(img, threshold, 255, cv2.THRESH_BINARY)

plt.subplot(1, 2, 1)
plt.title("Original Image")
plt.imshow(img, cmap='gray')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.title("Binary Image")
```

```
plt.imshow(binary_img, cmap='gray')
plt.axis('off')
```

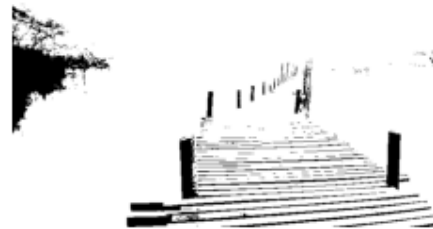
OUTPUT

Enter threshold value (0-255): 28
(np.float64(-0.5), np.float64(699.5), np.float64(465.5), np.float64(-0.5))<Figure size 640x480 with 2 Axes>

Original Image



Binary Image



QUESTION 2

WAP to convert rgb image to grayscale

SOLUTION

```
import numpy as np
import matplotlib.pyplot as plt
import cv2

img = cv2.imread("./rgb_original.jpg")
gray_img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

plt.subplot(1, 2, 1)
plt.title("Original Image")
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.axis('off')

plt.subplot(1, 2, 2)
plt.title("Grayscale Image")
plt.imshow(gray_img, cmap='gray')
plt.axis('off')
```

OUTPUT

(np.float64(-0.5), np.float64(899.5), np.float64(599.5), np.float64(-0.5))<Figure size 640x480 with 2 Axes>

Original Image



Grayscale Image



SOLUTION

```
import numpy as np
import matplotlib.pyplot as plt
import cv2

img = cv2.imread("./rgb_original.jpg")
gray_img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

r_weight = 0.7
g_weight = 0.2
b_weight = 0.1

weighted_gray = (r_weight * img[:, :, 2] + g_weight * img[:, :, 1] + b_weight * img[:, :, 0]).astype(np.uint8)

threshold = int(np.mean(weighted_gray))
_, binary_img = cv2.threshold(weighted_gray, threshold, 255, cv2.THRESH_BINARY)

plt.subplot(1, 3, 1)
plt.title("Original Image")
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.axis('off')

plt.subplot(1, 3, 2)
plt.title("Weighted Gray Image")
plt.imshow(weighted_gray, cmap='gray')
plt.axis('off')
```

OUTPUT

(np.float64(-0.5), np.float64(899.5), np.float64(599.5), np.float64(-0.5))<Figure size 640x480 with 2 Axes>

Original Image



Weighted Gray Image



QUESTION 3

WAP to draw a border around the input image, input the width (in terms of pixel count) and the desired color of the border from the user. Try this with a binary and grayscale image.

SOLUTION

```
import numpy as np
import matplotlib.pyplot as plt
import cv2

# WAP to draw a border around the input image, input the width (in terms of pixel count)
# and the desired color of the border from the user.
img = cv2.imread("./gray_original.png", cv2.IMREAD_GRAYSCALE)
border_width = int(input("Enter border width (in pixels): "))
border_color = int(input("Enter border color (0-255): "))

bordered_img = cv2.copyMakeBorder(img, border_width, border_width, border_width,
border_width, cv2.BORDER_CONSTANT, value=border_color)

plt.subplot(1, 2, 1)
plt.title("Original Image")
plt.imshow(img, cmap='gray')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.title("Image with Border")
plt.imshow(bordered_img, cmap='gray')
plt.axis('off')

plt.show()

threshold = int(np.mean(img))
_, binary_img = cv2.threshold(img, threshold, 255, cv2.THRESH_BINARY)
binary_img = cv2.copyMakeBorder(binary_img, border_width, border_width, border_width,
border_width, cv2.BORDER_CONSTANT, value=border_color)

plt.subplot(1, 2, 1)
plt.title("Original Image")
plt.imshow(img, cmap='gray')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.title("Binary Image")
plt.imshow(binary_img, cmap='gray')
plt.axis('off')

plt.show()
```

OUTPUT

```
Enter border width (in pixels): 80
Enter border color (0-255): 156
<Figure size 640x480 with 2 Axes><Figure size 640x480 with 2 Axes>
```

Original Image



Image with Border



Original Image



Binary Image



QUESTION 5

WAP for image complement (try with both, a grayscale and a binary image)

SOLUTION

```
import numpy as np
import matplotlib.pyplot as plt
import cv2

img = cv2.imread("./gray_original.png")
complement = 255 - img

binary_img = cv2.threshold(img, 128, 255, cv2.THRESH_BINARY)[1]
complement_binary = cv2.threshold(complement, 128, 255, cv2.THRESH_BINARY)[1]

plt.subplot(2, 2, 1)
plt.title("Original Gray Image")
plt.imshow(img, cmap='gray')
plt.axis('off')

plt.subplot(2, 2, 2)
plt.title("Complement Image")
plt.imshow(complement, cmap='gray')
plt.axis('off')

plt.subplot(2, 2, 3)
plt.title("Binary Image")
plt.imshow(binary_img, cmap='gray')
plt.axis('off')
```

```
plt.subplot(2, 2, 4)
plt.title("Complement Binary Image")
plt.imshow(complement_binary, cmap='gray')
plt.axis('off')
```

OUTPUT

(np.float64(-0.5), np.float64(699.5), np.float64(465.5), np.float64(-0.5))<Figure size 640x480 with 4 Axes>

Original Gray Image



Complement Image



Binary Image



Complement Binary Image



QUESTION 6

WAP to enhance the input image using log transform (assume $c = 1$)

SOLUTION

```
import numpy as np
import matplotlib.pyplot as plt
import cv2

img = cv2.imread("./rgb_original.jpg")

c = 255 / np.log(1 + float(np.max(img)))
# c = 1
log_img = np.uint8(c * np.log1p(img))

plt.subplot(1, 2, 1)
plt.title("Original Image")
plt.imshow(img)
plt.axis("off")

plt.subplot(1, 2, 2)
```

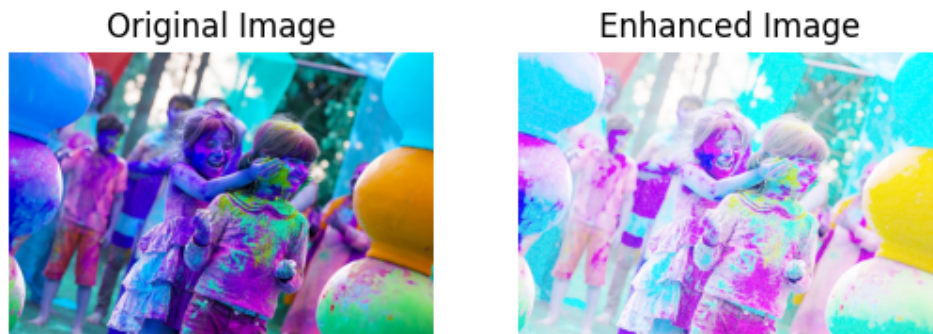


```
plt.title("Enhanced Image")
plt.imshow(log_img)
plt.axis("off")

plt.show()
```

OUTPUT

<Figure size 640x480 with 2 Axes>



Question 8: WAP to enhance image using power law/ gamma transformation

SOLUTION

```
import numpy as np
import matplotlib.pyplot as plt
import cv2

img = cv2.imread("./rgb_original.jpg")

gamma = float(input("Enter value of gamma: "))
c = 1.0

norm_img = img / 255

power_log_img = c * np.power(norm_img, gamma)

plt.subplot(1, 2, 1)
plt.title("Original Image")
plt.imshow(img)
plt.axis("off")

plt.subplot(1, 2, 2)
plt.title("After Law Transform")
plt.imshow(power_log_img)
plt.axis("off")
```

OUTPUT

Enter value of gamma: 0.5
(np.float64(-0.5), np.float64(899.5), np.float64(599.5), np.float64(-0.5))<Figure size 640x480 with 2 Axes>

Original Image



After Law Transform



QUESTION 9

WAP to enhance the input image using Intensity Level Slicing (Contrast Stretching)

SOLUTION

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("rgb_original.jpg").astype(np.float32)

r1, r2 = 80, 120
s1, s2 = 0, 100

mask = (img >= r1) & (img <= r2)

img[mask] = ((img[mask] - 1) / (r2 - r1)) * (s2 - s1) + s1
img = np.clip(img, 0, 255).astype(np.float32)

plt.subplot(1, 2, 1)
plt.title("Original Image")
plt.imshow(cv2.imread("rgb_original.jpg"))
plt.axis("off")

plt.subplot(1, 2, 2)
plt.title("After Contrast Stretching")
plt.imshow(img)
plt.axis("off")
```

OUTPUT

Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..255.0].
(np.float64(-0.5), np.float64(899.5), np.float64(599.5), np.float64(-0.5))<Figure size 640x480 with 2 Axes>

Original Image



After Contrast Stretching



QUESTION 10

WAP to enhance the input image using histogram equalization

SOLUTION

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("./gray_original.png", cv2.IMREAD_GRAYSCALE)

equalized_img = cv2.equalizeHist(img)

plt.subplot(1, 2, 1)
plt.title("Original Image")
plt.imshow(img, cmap='gray')
plt.axis("off")

plt.subplot(1, 2, 2)
plt.title("After Histogram Equalization")
plt.imshow(equalized_img, cmap='gray')
plt.axis("off")
```

OUTPUT

(np.float64(-0.5), np.float64(699.5), np.float64(465.5), np.float64(-0.5))<Figure size 640x480 with 2 Axes>

Original Image



After Histogram Equalization



QUESTION 11

WAP to match the histogram of the input image with that of reference image using histogram matching technique

SOLUTION

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

def histogram_matching(source, reference):
    src_hist, bins = np.histogram(source.flatten(), 256, [0, 256])
    ref_hist, bins = np.histogram(reference.flatten(), 256, [0, 256])

    src_cdf = src_hist.cumsum()
    ref_cdf = ref_hist.cumsum()

    src_cdf_normalized = src_cdf * (255 / src_cdf[-1])
    ref_cdf_normalized = ref_cdf * (255 / ref_cdf[-1])

    lookup_table = np.zeros(256)
    ref_idx = 0
    for src_idx in range(256):
        while (
            ref_idx < 255 and ref_cdf_normalized[ref_idx] < src_cdf_normalized[src_idx]
        ):
            ref_idx += 1
        lookup_table[src_idx] = ref_idx

    matched_img = cv2.LUT(source, lookup_table.astype(np.uint8))

    return matched_img

source_img = cv2.imread("./gray_original.png", cv2.IMREAD_GRAYSCALE)
reference_img = cv2.imread("./rgb_original.jpg", cv2.IMREAD_GRAYSCALE)
matched_img = histogram_matching(source_img, reference_img)

plt.subplot(1, 3, 1)
plt.title("Source Image")
plt.imshow(source_img, cmap="gray")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.title("Reference Image")
plt.imshow(reference_img, cmap="gray")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.title("Matched Image")
plt.imshow(matched_img, cmap="gray")
plt.axis("off")
```

OUTPUT

(np.float64(-0.5), np.float64(699.5), np.float64(465.5), np.float64(-0.5))<Figure size 640x480 with 3 Axes>

Source Image



Reference Image



Matched Image

**QUESTION 12**

WAP to smooth the image using Averaging filter (un-weighted), Weighted filter given by $h(x, y) = \max(|x|, |y|)$, gaussian filter

SOLUTION

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("./rgb_original.jpg")

# Averaging filter
kernel_size = 5
averaging_kernel = np.ones((kernel_size, kernel_size), np.float32) / (
    kernel_size * kernel_size
)
averaged_img = cv2.filter2D(img, -1, averaging_kernel)

# TODO: Weighted filter given by  $h(x, y) = \max(|x|, |y|)$ 
weighted_kernel = np.array([[1, 2, 1], [2, 4, 2], [1, 2, 1]], np.float32) / 16
weighted_img = cv2.filter2D(img, -1, weighted_kernel)

# Gaussian filter
gaussian_img = cv2.GaussianBlur(img, (5, 5), 0)

plt.subplot(1, 4, 1)
plt.title("Original Image")
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.axis("off")

plt.subplot(1, 4, 2)
plt.title("Averaging Filter")
plt.imshow(cv2.cvtColor(averaged_img, cv2.COLOR_BGR2RGB))
plt.axis("off")

plt.subplot(1, 4, 3)
plt.title("Weighted Filter")
plt.imshow(cv2.cvtColor(weighted_img, cv2.COLOR_BGR2RGB))
plt.axis("off")

plt.subplot(1, 4, 4)
plt.title("Gaussian Filter")
plt.imshow(cv2.cvtColor(gaussian_img, cv2.COLOR_BGR2RGB))
plt.axis("off")
```

OUTPUT

(np.float64(-0.5), np.float64(899.5), np.float64(599.5), np.float64(-0.5))<Figure size 640x480 with 4 Axes>

Original Image Averaging Filter Weighted Filter Gaussian Filter



QUESTION 13

WAP to sharpen the input image using Laplacian filter, Gradient operators (Sobel, Canny), Unsharp masking

SOLUTION

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("./rgb_original.jpg")
gray_img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Laplacian filter
laplacian_img = cv2.Laplacian(gray_img, cv2.CV_64F)

# Sobel operator
sobel_x = cv2.Sobel(gray_img, cv2.CV_64F, 1, 0, ksize=5)
sobel_y = cv2.Sobel(gray_img, cv2.CV_64F, 0, 1, ksize=5)
sobel_img = cv2.magnitude(sobel_x, sobel_y)

canny_img = cv2.Canny(gray_img, 100, 200)

# Unsharp masking
blurred_img = cv2.GaussianBlur(gray_img, (5, 5), 0)
unsharp_img = cv2.addWeighted(gray_img, 1.5, blurred_img, -0.5, 0)
unsharp_img = np.clip(unsharp_img, 0, 255).astype(np.uint8)

plt.subplot(2, 2, 1)
plt.title("Original Image")
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.axis("off")

plt.subplot(2, 2, 2)
plt.title("Laplacian Filter")
plt.imshow(laplacian_img, cmap="gray")
plt.axis("off")

plt.subplot(2, 2, 3)
plt.title("Sobel Operator")
plt.imshow(sobel_img, cmap="gray")
plt.axis("off")

plt.subplot(2, 2, 4)
plt.title("Canny Edge Detection")
plt.imshow(canny_img, cmap="gray")
plt.axis("off")
```

OUTPUT

(np.float64(-0.5), np.float64(899.5), np.float64(599.5), np.float64(-0.5))<Figure size 640x480 with 4 Axes>

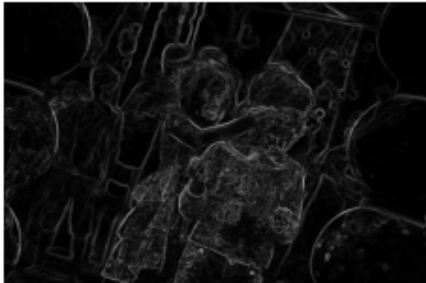
Original Image



Laplacian Filter



Sobel Operator



Canny Edge Detection

